STUDY NOTES

Hugging Face LLM Course

Chapter 1 — NLP, Transformers & Large Language Models

A structured reference covering NLP fundamentals, the pipeline API, Transformer architecture and history, transfer learning, attention, encoder/decoder model families, and bias & limitations — with a cheat sheet, glossary, interview questions, quiz, and hands-on exercises.

Hugging Face LLM Course — Chapter 1

NLP, Transformers & Large Language Models — Study Notes

> Concise reference notes covering NLP fundamentals, the `pipeline()` API, Transformer architecture, transfer learning, attention, encoder/decoder model families, and bias & limitations.

Table of Contents

---

1. NLP & Large Language Models

OVERVIEW

NLP (Natural Language Processing) is the broad field concerned with getting computers to understand, interpret, and generate human language. LLMs (Large Language Models) are the current dominant tool for that job — large neural networks trained on massive amounts of text that can handle many language tasks without being purpose-built for just one.

WHY IT MATTERS

Separating “the field” (NLP) from “the current best tool in the field” (LLMs) keeps the mental model clean. Classic NLP tasks — classification, entity recognition, question answering, translation — are still the vocabulary used to describe what an LLM is doing. LLMs didn’t replace the task list; they changed how the tasks on that list get solved.

CORE CONCEPTS

- Human language is ambiguous and context-dependent; machines need it converted to numbers before anything useful can happen. - Classic NLP tasks: classifying whole sentences (sentiment, spam detection, grammatical correctness), classifying individual words (part-of-speech, named entities), generating text, filling in masked words, answering questions, and translating between languages. - An LLM is defined by three traits: it’s **large** (parameters + training data), it’s a **language model** (predicts/scores sequences of text), and it’s **general-purpose** (one model, many tasks, usable via prompting or fine-tuning instead of being rebuilt per task). - **Emergent abilities**: capabilities that appear reliably only once a model crosses a certain scale threshold — not explicitly programmed in, and largely absent in smaller versions of the same model family. - Even with LLMs, ambiguity, sarcasm, and deep cultural context remain genuinely hard problems. Scale helps a lot; it doesn’t fully solve understanding.

KEY TAKEAWAY

NLP is the “what” (the problems); LLMs are the current “how” (the solution). Before LLMs, each task typically needed its own specialized model. A single sufficiently large pretrained model can now be prompted or lightly adapted for sentiment analysis, translation, summarization, and open-ended chat — because next-token prediction at scale forces the model to implicitly learn grammar, facts, and reasoning patterns along the way.

EXAMPLE

A customer support inbox used to need separate models for spam detection, topic tagging, sentiment, and auto-reply drafting. A modern setup can run all four off a single LLM through different prompts — fewer models to maintain, at the cost of needing solid prompt and evaluation design.

```
NLP  = the FIELD   (the problems: classify, extract, translate, generate...)
LLM  = the current best TOOL for solving most of that field's problems

[ text in ] --> [ NLP task: classification / QA / NER / translate ]
                          |
                          v
        solved today mostly by --> [ an LLM ]
```

Why “large” is the operative word (illustrative, order-of-magnitude parameter counts):

Model	Year	Approx. parameters

BERT-base	2018	~110M
GPT-2	2019	~1.5B
GPT-3	2020	~175B
Modern LLMs	2023+	100B–1T+

QUICK REVISION POINTS

- NLP = the field; LLM = the dominant current technique. - Language is ambiguous — machines need it converted to numbers to do anything useful. - Classic task list still applies: classification, NER, QA, translation, summarization, generation. - LLM = large + language model (predicts text) + general-purpose. - Emergent abilities appear at scale, not by explicit design. - LLMs reduce — but don't eliminate — hard problems like sarcasm and cultural nuance.

COMMON MISTAKES

- Treating "NLP" and "LLM" as two unrelated technologies instead of field-vs-tool. - Assuming a bigger model automatically fixes every weakness — scale mainly improves aggregate capability, not every specific failure mode. - Forgetting that "general-purpose" doesn't mean "perfect at everything out of the box" — per-task evaluation still matters.

---

2. What Can Transformers Do? — The Pipeline API

OVERVIEW

The `pipeline()` helper is the fastest way into the 🤗 Transformers library — it wraps tokenizing text, running the model, and turning raw output into something readable, all behind one call.

WHY IT MATTERS

Pipelines are the quickest way to sanity-check whether an off-the-shelf pretrained model is in the right ballpark for a problem before investing in a custom setup or fine-tuning. They also model what inference actually consists of: preprocess → model → postprocess, every time.

CORE CONCEPTS

- Three-stage flow, always: **tokenizer** (text → numbers) → **model** (numbers → raw scores/logits) → **postprocessing** (raw scores → readable output). - The `pipeline()` function isn't limited to text — it also supports images, audio, and multimodal tasks — but this chapter focuses on text. - Task list to recognize: sentiment analysis, zero-shot classification, text generation, fill-mask, named entity recognition (NER), extractive question answering, summarization, translation. - **Zero-shot classification** is the standout case — you can hand it label names invented on the spot, and it still works because it treats classification as "does this text logically imply this label?" - A default model is auto-picked per task if none is specified, but any compatible checkpoint from the Hub can be swapped in.

KEY TAKEAWAY

A pipeline is a thin, convenient wrapper — not a "lesser" version of the model. Under the hood it's the same tokenizer and model objects you'd wire up manually; the pipeline just removes the boilerplate. Once the three-stage flow clicks, every task pipeline is "the same shape," just with a different postprocessing function at the end.

Example

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
classifier("I've been waiting for a HuggingFace course my whole life.")
# [{'label': 'POSITIVE', 'score': 0.9598}]

generator = pipeline("text-generation", model="HuggingFaceTB/SmolLM2-360M")
generator("In this course, we will teach you how to", max_length=30, num_return_sequences=2)
```

Prototyping a moderation feature: reach for `pipeline("text-classification")` with a general toxicity model first, to see if it's "good enough" before deciding whether a custom fine-tuned classifier is needed at all.

```
Raw Text           Tokenizer              Model                   Postprocess
"I loved it"  -->  text → token IDs  -->  Transformer forward  -->  logits → label/score
                                         pass → raw logits
```

Task	What it does	Typical use
`sentiment-analysis`	Classifies polarity of text	Review/feedback triage
`zero-shot-classification`	Classifies against labels never explicitly trained on	Fast prototyping, dynamic taxonomies
`text-generation`	Continues a prompt	Drafting, autocomplete
`fill-mask`	Predicts a hidden word in a sentence	Understanding-style pretraining task
`ner`	Tags entities (person, org, location…)	Extraction from documents
`question-answering`	Extracts an answer span from a passage	Search / doc Q&A
`summarization`	Produces a shorter version of a text	Long-doc digestion
`translation`	Converts text between languages	Localization

DEVELOPER NOTES

- Don't re-create a pipeline object per request in a server — loading the model/tokenizer is the expensive part; instantiate once, reuse. - Always check which model a pipeline defaults to for a given task; defaults can change across library versions and might not be the best fit for a specific

domain or language.

QUICK REVISION POINTS

- Pipeline = tokenizer + model + postprocessing, bundled behind one call. - Same three-stage flow underlies every task. - Zero-shot classification reframes the task as entailment — no retraining needed for new labels. - Pipelines are convenience, not a “toy” — same underlying production-grade models as custom code. - Great for quick prototyping and baselining before investing in fine-tuning.

COMMON MISTAKES

- Assuming pipelines aren’t “production-ready” — they wrap the same production-grade models. - Instantiating a new pipeline on every request (kills latency/throughput). - Not checking the default model before shipping a demo built on defaults.

> **Interview tip:** If asked “what happens inside `pipeline()`,” name the three stages explicitly — interviewers are checking whether the abstraction is understood as a facade, not magic.

---

3. How Transformers Work — History & Big Picture

OVERVIEW

The Transformer is the neural network architecture (introduced in June 2017) behind essentially every modern LLM. Its defining trait: it relates different words in a sequence to each other using **attention**, instead of processing the sequence strictly one word at a time like older recurrent models did.

WHY IT MATTERS

Understanding *why* the Transformer replaced RNNs/LSTMs explains almost every downstream design choice in modern NLP systems — training speed, context length limits, and why encoder/decoder splits exist at all.

CORE CONCEPTS

- **Old approach (RNN/LSTM):** process the sequence step by step, carrying a hidden state forward. Two big problems: training can’t be parallelized across time steps, and information from early tokens decays over long sequences. - **Transformer fix:** attention lets every token look directly at every other token in one step — no decay, and no forced sequential dependency within a layer, so training parallelizes well on GPUs. - **Timeline:** - **June 2017** — Transformer introduced, focused on machine translation (encoder + decoder). - **June 2018** — GPT, the first pretrained Transformer used for fine-tuning across NLP tasks (decoder-only). - **October 2018** — BERT, a large pretrained model designed to produce better sentence representations (encoder-only). - **February 2019** — GPT-2, a bigger, improved GPT, initially withheld from public release over misuse concerns. - **2019+** — an explosion of variants (T5, BART, DistilBERT, and more). - **Today** — most large-scale general-purpose LLMs are decoder-only. - The original architecture = **encoder** (reads and understands input) + **decoder** (generates output), connected together — built for machine translation.

KEY TAKEAWAY

Transformers swapped “pass information down a long chain” for “let every token talk to every other token directly.” That single change is why training got faster (parallel, not sequential) and why long documents stopped losing context so badly.

EXAMPLE

Translating “You like this course” to French requires looking back at “You” to get the correct verb form for “like.” An RNN has to carry that information forward through every intermediate step; a Transformer lets the verb’s representation directly attend back to “You” — one hop, regardless of distance.

```
Timeline
2017 ── 2018 (Jun) ── 2018 (Oct) ── 2019+ ──────────── Today
Transformer   GPT          BERT      BART / T5 /    Modern LLMs
(encoder+     (decoder-    (encoder-  DistilBERT...  (mostly
 decoder,      only,        only,     (variant       decoder-only,
 translation)  pretrain+    masked    explosion)     general-purpose)
               finetune)    LM)

RNN — sequential, one step at a time
w1 -> state -> w2 -> state -> w3 -> w4
Long chain: context can decay, and steps can't run in parallel.

Transformer — every token attends to every other token directly
w1   w2   w3   w4      (all connected — no decay, fully parallel)
```

```
       ENCODER (× N)                          DECODER (× N)
Input Embedding + Positional Encoding   Output Embedding + Positional Encoding
Multi-Head Self-Attention               Masked Self-Attention
Feed-Forward Network                    Cross-Attention (keys & values from encoder)
                                         Feed-Forward Network
                                         Linear + Softmax
"You" "like" "this" "course"  ------->  "Tu" "aimes" "ce" "cours" (generated so far)
```

DEVELOPER NOTES

- Attention cost grows quadratically with sequence length — this is the real reason “long context” is a hard systems problem, not just a “use a bigger model” problem. - When reading a new open-source model’s card, the first thing worth checking is whether it’s encoder-only, decoder-only, or encoder-decoder — that single fact predicts most of what it’s good at.

QUICK REVISION POINTS

- RNNs: sequential, slow to train, long-range info decays. - Transformers: attention-based, parallelizable, no decay over distance. - 2017 Transformer → 2018 GPT (decoder) & BERT (encoder) → today’s LLMs mostly decoder-only. - Original design = encoder + decoder, built for

translation. - “Attention Is All You Need” is literally the founding paper’s title — attention is the core idea, not a minor add-on.

COMMON MISTAKES

- Thinking “Transformer” and “LLM” are always the same thing — small task-specific Transformers exist and aren’t “LLMs” in the broad sense. - Assuming the switch to attention was purely about speed — long-range dependency handling was just as important a motivation.

---

4. Transfer Learning — Pretraining vs. Fine-Tuning

OVERVIEW

Transfer learning means reusing a model that already learned general language skills from huge amounts of text (**pretraining**), instead of training a fresh model from scratch for every new task. Optionally, that pretrained model gets further adapted on a smaller, task-specific dataset (**fine-tuning**).

WHY IT MATTERS

This is the economic engine behind the whole field: pretraining is extremely expensive and done rarely, by well-resourced labs; fine-tuning is comparatively cheap and is what most engineers and teams actually do.

CORE CONCEPTS

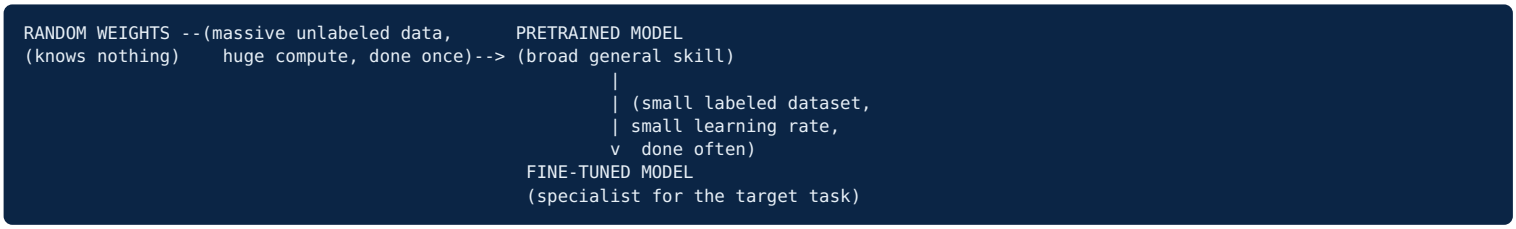
- **Pretraining:** train from random weights on a huge unlabeled text corpus using a self-supervised objective (predict the next word, or predict a hidden word) — no human labeling needed, so it scales to internet-sized data. - **Fine-tuning:** continue training a pretrained model on a smaller, often labeled dataset, usually with a much smaller learning rate, to specialize it for one task. - Fine-tuning is cheap and fast specifically because it starts from an already-good set of weights instead of random ones. - **Risk:** too much or too aggressive fine-tuning can cause **catastrophic forgetting** — the model loses general skills it had right after pretraining.

KEY TAKEAWAY

Think of pretraining as “general education” and fine-tuning as “a short apprenticeship.” The apprenticeship is short and cheap only because the person already knows how to read, reason, and write — fine-tuning only has to teach the gap between general competence and the specific job, not competence from zero.

EXAMPLE

Taking a general pretrained encoder and fine-tuning it on a few thousand labeled support tickets to classify urgency takes a fraction of the time and data that training a ticket classifier from scratch would need.



DEVELOPER NOTES

- Before fine-tuning anything, try prompting the pretrained/instruction-tuned model first — sometimes that’s already good enough and saves the whole fine-tuning effort. - If fine-tuning does happen, keep a small “general capability” eval running alongside the task metric, specifically to catch catastrophic forgetting early.

QUICK REVISION POINTS

- Pretraining = expensive, rare, unlabeled data, general skill. - Fine-tuning = cheap, common, smaller/labeled data, specialized skill. - Self-supervised objective = the label comes from the data itself (next word / masked word). - A smaller learning rate in fine-tuning avoids destroying general knowledge. - Catastrophic forgetting = the main risk of over-aggressive fine-tuning.

COMMON MISTAKES

- Assuming fine-tuning always means retraining every parameter — lightweight, adapter-style fine-tuning is common and often preferable. - Jumping straight to fine-tuning without first testing whether prompting alone is sufficient. - Using too high a learning rate during fine-tuning “to make it learn faster” — a fast way to trigger catastrophic forgetting.

> **Tip:** Always benchmark the plain pretrained model with good prompting before writing a single line of fine-tuning code. It’s the cheapest experiment available, and it sets the baseline everything else has to beat.

---

5. Attention — The Core Trick

OVERVIEW

Attention layers let a model decide, for each word, which other words in the sentence matter most to it right now — and weight them accordingly — instead of treating every other word as equally relevant.

WHY IT MATTERS

This is the mechanism that actually makes “understanding context” possible. Without it, a model has no principled way to know that in “The trophy didn’t fit in the suitcase because it was too big,” the word “it” refers to the trophy, not the suitcase.

CORE CONCEPTS

- Attention computes a relevance score between the current word and every other word in context, then builds a weighted blend that leans heavily

on the most relevant ones. - Classic translation example: producing the right form of “like” for “You like this course” requires attending back to “You” — attention is literally what lets the model do that “looking back.” Likewise, translating “this” requires attending forward to “course,” since the article depends on the noun’s gender. - The **encoder** can look both directions (before and after a word); the **decoder**, during generation, can only look at words it has already produced — it can’t peek at the future. - This is intentionally a high-level pass; the underlying math (Query/Key/Value) gets more technical in later chapters.

KEY TAKEAWAY

Attention is a learned spotlight. For every word, it asks “which other words should I be paying attention to right now?” and produces a weighted mix leaning toward the useful ones. That mix becomes the word’s new, context-aware representation.

EXAMPLE

Long-document summarization: a fact stated in paragraph one can directly influence a sentence generated near the end, because attention gives it a direct path — it doesn’t have to survive being passed through every paragraph in between.

```
"You like this course" — arc thickness = attention weight

  You -------- like        this -------- course
        "like" attends strongly back to "You"
           (subject → verb form)

        "this" attends strongly forward to "course"
        (to pick masculine/feminine article in French)
```

DEVELOPER NOTES

- When a model mishandles a long input, first suspect whether the relevant detail sits somewhere attention doesn’t weight well for that input length — reordering critical info closer to the end of the prompt is a cheap mitigation worth trying. - Raw attention-weight visualizations are useful signal, not a complete explanation of “why” a model did something.

QUICK REVISION POINTS

- Attention = learned relevance weighting across words in context. - Encoder attention: bidirectional (sees both directions). - Decoder attention (during generation): can only see what’s already been generated. - Attention is the mechanism that solves “which earlier word does this word depend on.” - The founding paper’s title says it directly: attention is the central idea of the architecture.

COMMON MISTAKES

- Thinking attention is a minor add-on feature rather than the architecture’s core mechanism. - Assuming attention weights are a complete, reliable explanation of model reasoning.

> **Exam/interview tip:** If asked “why do we need attention at all,” lead with a concrete example — pronoun resolution or verb agreement across a sentence. Concrete examples land far better than reciting the definition.

---

6. Encoder / Decoder / Encoder-Decoder Models

OVERVIEW

The Transformer splits cleanly into two halves — an encoder that builds understanding of input, and a decoder that generates output. Depending on the task, models use just one half or both.

WHY IT MATTERS

This is the single most useful fact for picking the right model family for a project. Getting this mapping wrong usually means overpaying (a huge generative model for a simple classification task) or underdelivering (a classification-only model for something needing open-ended generation).

CORE CONCEPTS

- **Encoder-only:** builds a rich representation of input using context from both directions. Good for understanding tasks — sentence classification, NER. Example: BERT. - **Decoder-only:** generates output step by step, each step conditioned on what came before (causal/left-to-right attention). Good for generative tasks — text generation, chat. Example: GPT. - **Encoder-decoder (seq2seq):** the encoder fully understands the input first, then the decoder generates a genuinely different output sequence, conditioned on that understanding, via cross-attention. Good for translation and summarization. Example: T5, BART. - Each half can be used independently depending on what the task actually needs.

KEY TAKEAWAY

A simple mental shortcut: “Do I just need to understand or label something that already exists?” → encoder. “Do I need to produce brand-new, open-ended text?” → decoder. “Do I need to transform one sequence into a meaningfully different one?” → encoder-decoder.

EXAMPLE

A resume-screening classifier → encoder-only. A code-completion tool → decoder-only. A translation feature → encoder-decoder (or a large decoder-only LLM prompted to translate — a different cost/flexibility trade-off).

```
ENCODER-ONLY            DECODER-ONLY            ENCODER-DECODER
  Bidirectional           Causal                  Enc  ->  Dec
  Encoder                 Decoder                 e.g. T5 / BART
  e.g. BERT               e.g. GPT                translate · summarize
  classify · extract ·    generate · chat ·
  embed                   complete
```

Family	Attention style	Best for	Example
Encoder-only	Bidirectional	Classification, NER, extraction	BERT

Decoder-only	Causal (left-to-right only)	Open-ended generation, chat	GPT
Encoder-decoder	Bidirectional + causal + cross-attention	Translation, summarization	Original Transformer, T5, BART

DEVELOPER NOTES

- For high-volume, narrow tasks (classification/extraction), a small fine-tuned encoder-only model is usually far cheaper and faster in production than prompting a large decoder-only LLM for the same job. - Semantic search / RAG pipelines almost always use an encoder-only model for the embedding step, even when the “answering” model is a large decoder-only LLM.

QUICK REVISION POINTS

- Encoder-only = understanding tasks, bidirectional context. - Decoder-only = generation tasks, left-to-right only. - Encoder-decoder = transformation tasks (translation, summarization). - Task shape (understand vs. generate vs. transform) should drive architecture choice. - Most of today’s general-purpose LLMs are decoder-only at large scale.

COMMON MISTAKES

- Reaching for a giant decoder-only LLM by default, even for simple classification tasks a small encoder-only model would handle more cheaply and just as accurately. - Forgetting that encoder-decoder ≠ “two unrelated models glued together” — the decoder specifically attends back into the encoder’s output at every generation step.

> **Recap:** Understand → Encoder. Generate → Decoder. Transform → Encoder-Decoder.

---

7. Bias & Limitations

OVERVIEW

Pretrained models learn whatever statistical patterns exist in their training data — including stereotypes, factual errors, and imbalanced representation. This isn’t a bug to patch out; it’s a structural side effect of learning from real-world text.

WHY IT MATTERS

This directly affects hiring tools, moderation systems, and anything that produces or ranks content about people. It can’t be responsibly ignored when shipping an LLM-backed feature.

CORE CONCEPTS

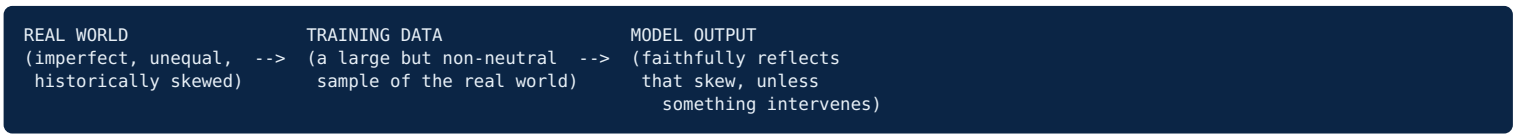
- Training data reflects the real world’s imbalances — the training objective (predict the most likely next/masked word) has no built-in notion of “fair” vs. “unfair” pattern; it just learns what’s statistically present. - Even a very capable, fluent model can produce biased associations or stereotyped completions without any obvious “bug” in the code. - Fine-tuning and careful prompt/product design can reduce, but not fully eliminate, learned bias. - **Fluent ≠ correct** — a model can generate confident, well-written, factually wrong content. This is a separate limitation, known as **hallucination**.

KEY TAKEAWAY

A model is a mirror of its training data, not an independent judge of fairness or truth. If the data was skewed, the model’s default behavior will be skewed too, unless someone deliberately intervenes.

EXAMPLE

A generic sentiment or generation model asked to write about “a nurse” and “a doctor” might default to gendered assumptions for each role — purely because that pattern was common in its training data, not because anyone intentionally coded that behavior in.



DEVELOPER NOTES

- Never ship an “our model is unbiased” claim without specifics — ask what mitigation was actually applied and what evaluation was run, on data that matches the real use case. - For anything decision-adjacent about people (hiring, lending, moderation), plan an explicit bias/fairness evaluation step, not just a generic accuracy metric.

QUICK REVISION POINTS

- Bias comes primarily from training data, not from a specific “bug.” - The training objective has no built-in fairness or truth constraint. - Bigger models don’t automatically become less biased. - Mitigation is ongoing (data, training, guardrails, monitoring) — not a single fix. - Fluent output is not the same thing as correct or unbiased output.

COMMON MISTAKES

- Assuming a bigger/newer model version automatically means “less biased.” - Treating bias mitigation as a one-time checklist item instead of an ongoing process. - Conflating “the model sounds confident” with “the model is correct.”

---

8. Chapter Summary

The whole chapter in five lines 1. NLP is the field; LLMs are today’s dominant technique for solving NLP tasks at scale. 2. The `pipeline()` API shows the universal shape of inference: tokenize → model → postprocess. 3. Transformers replaced RNNs by using attention instead of sequential recurrence — faster to train, no long-range decay. 4. Transfer learning (pretrain once, expensively; fine-tune often, cheaply) is the economic strategy that makes all of this practical. 5. Model shape follows task shape — encoder = understand, decoder = generate, encoder-decoder =

transform — and none of this removes the need to watch for bias and factual limitations.

What clicks - Attention = direct, one-hop relevance between any two words, regardless of distance. - Fine-tuning is cheap specifically because pretraining already did the expensive part. - Architecture choice should be driven by task shape, not by “which model is trendiest.”

Worth digging into further - The literal math inside attention (Query/Key/Value) — this chapter stays conceptual. - How tokenizers actually split words into subwords. - A concrete fine-tuning workflow with real code, not just the concept.

---

9. Cheat Sheet

```
NEED TO...                          REACH FOR...
-------------------------------------------------------------
Classify a piece of text           ->  Encoder-only model
Extract entities/spans             ->  Encoder-only model
Get an embedding for search        ->  Encoder-only model
Generate open-ended text / chat    ->  Decoder-only model
Translate / summarize              ->  Encoder-decoder model
                                        (or a prompted decoder-only LLM)

PRETRAIN  = learn general language skill from massive unlabeled text
            (expensive, rare, self-supervised)
FINE-TUNE = adapt pretrained model to one task with smaller data
            (cheap, common, smaller learning rate)

ATTENTION (plain-English version):
  for each word -> score relevance to every other word in context
                -> blend context weighted by those relevance scores
```

Question to ask	Answer tells you
Does this task need me to just understand/label existing text?	Encoder-only
Does this task need me to produce brand-new open-ended text?	Decoder-only
Does this task transform one sequence into a different one?	Encoder-decoder
Is there little/no labeled data for this exact task?	Try prompting a pretrained model first
Is it a clear, narrow, high-volume task?	Consider fine-tuning a small model

---

10. Terminology

Term	Working definition
Token	A small chunk of text (often a subword) that a model actually processes, produced by a tokenizer.
Embedding	A dense vector representing a token/word such that similar meanings end up numerically close together.
Parameter	A learned numeric weight inside the model; “model size” usually refers to total parameter count.
Context window	The maximum number of tokens a model can consider at once (input + generated output).
Attention	The mechanism that computes relevance-weighted relationships between tokens in a sequence.
Encoder	The half of a Transformer that builds a bidirectional understanding of input.
Decoder	The half of a Transformer that generates output sequentially, left-to-right.
Pretraining	Training a model from scratch on massive unlabeled text with a self-supervised objective.
Fine-tuning	Continuing training a pretrained model on a smaller, task-specific dataset.
Zero-shot	Performing a task with no task-specific labeled training examples, usually via prompting.
Emergent ability	A capability that appears reliably only once a model passes a certain scale.
Hallucination	Fluent, confident output that is factually wrong or unsupported.
Bias (model)	Systematic skew in outputs reflecting imbalances present in training data.
Pipeline	A high-level API bundling tokenizer + model + postprocessing behind one call.

---

11. Acronyms

Acronym	Stands for

LLM	Large Language Model
NLP	Natural Language Processing
NLU	Natural Language Understanding
NLG	Natural Language Generation
GPU	Graphics Processing Unit
TPU	Tensor Processing Unit
Transformer	(Architecture name, not an acronym) — attention-based sequence model, 2017
Token	(Not an acronym) — smallest text unit a model processes
Parameter	(Not an acronym) — a learned weight in the model
Context Window	(Not an acronym) — max tokens a model can attend to at once
BERT	Bidirectional Encoder Representations from Transformers
GPT	Generative Pretrained Transformer
NER	Named Entity Recognition
QA	Question Answering
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
Seq2Seq	Sequence-to-Sequence
API	Application Programming Interface

12. Interview Questions (20)

1. **What is the difference between NLP and an LLM?** NLP is the broad field of processing human language with computers; an LLM is a specific, currently dominant type of model (large, Transformer-based, general-purpose) used to solve NLP tasks.
2. **What three stages does every inference pipeline perform?** Tokenization/preprocessing, the model's forward pass, and postprocessing into a task-appropriate output.
3. **Why does zero-shot classification work without task-specific training?** It reframes classification as an entailment problem ("does the text imply this label?"), which the model already knows how to judge generally.
4. **What two limitations of RNNs did the Transformer address?** Lack of parallelism during training (sequential computation) and decay of long-range dependencies over long sequences.
5. **What does "Attention Is All You Need" refer to?** The title of the 2017 paper that introduced the Transformer architecture, centered on attention instead of recurrence.
6. **What's the difference between an encoder and a decoder in a Transformer?** The encoder builds a bidirectional representation of the input; the decoder generates output sequentially, attending only to already-produced tokens (plus the encoder's output, if paired with one).
7. **When would you use an encoder-only model instead of a decoder-only model?** When the task is understanding-only (classification, NER, extraction) rather than open-ended generation.
8. **When would you use an encoder-decoder model?** For tasks that transform an input sequence into a meaningfully different output sequence, like translation or summarization.
9. **What is pretraining?** Training a model from scratch on a massive unlabeled corpus using a self-supervised objective (e.g., predict the next/masked token) to learn general language competence.
10. **What is fine-tuning, and why is it usually cheap?** Continuing to train a pretrained model on a smaller, task-specific dataset; it's cheap because it starts from an already-good set of weights instead of random initialization.
11. **What is catastrophic forgetting?** The loss of general capabilities a pretrained model had, caused by fine-tuning too aggressively (e.g., too high a learning rate or too much task-specific data pushing the weights too far).
12. **What is an emergent ability?** A capability that reliably appears only once a model reaches a certain scale, and is largely absent in smaller models trained the same way.
13. **Why can't a purely rule-based system handle language well at scale?** Language is ambiguous, essentially infinite in its possible sentences, and constantly evolving — rules can't be hand-written to cover all of it.
14. **What causes bias in a pretrained model?** The model learns whatever statistical patterns exist in its training data, including real-world imbalances and stereotypes; the standard training objective has no built-in fairness constraint.
15. **What is hallucination, and how is it different from bias?** Hallucination is confidently stated but factually incorrect output; bias is systematic skew in how the model represents or treats different groups/topics. Different causes, different mitigations.
16. **Does scaling a model up always reduce bias?** No — scale reliably improves many aggregate capability metrics but doesn't guarantee reduced bias, and can even make biased output more fluently confident.
17. **What is self-supervised learning, and why does pretraining rely on it?** Learning where labels come automatically from the data itself (e.g., the next word is the label). Pretraining relies on it because it removes the human-labeling bottleneck, allowing training at massive scale.
18. **What does "general-purpose" mean in the context of an LLM?** The same model can be applied to many different tasks (classification, generation, translation, etc.) without being rebuilt from scratch for each one.
19. **Give an example where attention is clearly necessary for correct output.** Resolving "it" in "The trophy didn't fit in the suitcase because it was too big" — correctly requires attending back to "trophy," not "suitcase."
20. **Why might a team choose a small fine-tuned encoder model over a large decoder-only LLM API for a classification feature?** Lower

latency and cost per request at scale, more predictable behavior, and easier evaluation for a narrow, well-defined understanding task.

---

13. Mini Quiz — 20 MCQs

1. What best describes the relationship between NLP and LLMs?
   A. They are unrelated fields B) LLM is the field, NLP is a tool C) NLP is the field, LLM is a dominant current technique D) They are exact synonyms **Answer: C**
2. Which stage is NOT part of the standard pipeline flow?
   A. Tokenization B) Model forward pass C) Postprocessing D) Manual regex rule writing **Answer: D**
3. Zero-shot classification works by:
   A. Training a new model per label B) Reframing classification as entailment C) Using only keyword matching D) Random guessing among labels **Answer: B**
4. What core limitation of RNNs did Transformers solve?
   A. Too much parallelism B) Sequential computation and long-range decay C) Too few parameters D) Inability to use GPUs at all **Answer: B**
5. The Transformer architecture was introduced in:
   A. 2015 B) 2017 C) 2020 D) 2022 **Answer: B**
6. GPT is best described as:
   A. Encoder-only B) Decoder-only C) Encoder-decoder D) Not a Transformer **Answer: B**
7. BERT is best described as:
   A. Encoder-only B) Decoder-only C) Encoder-decoder D) A rule-based system **Answer: A**
8. Which task is a natural fit for an encoder-decoder model?
   A. Sentiment classification B) Named entity recognition C) Machine translation D) Text embedding for search **Answer: C**
9. Pretraining typically uses:
   A. Small labeled datasets B) Massive unlabeled text with a self-supervised objective C) Manual rule sets D) No data at all **Answer: B**
10. Fine-tuning is usually cheaper than pretraining because:
    A. It uses more compute B) It starts from already-good weights, not random ones C) It ignores the task entirely D) It requires more data than pretraining **Answer: B**
11. Catastrophic forgetting refers to:
    A. A GPU running out of memory B) Losing general capability due to overly aggressive fine-tuning C) A tokenizer bug D) A model refusing to answer **Answer: B**
12. An emergent ability is:
    A. Present in all model sizes equally B) Explicitly hand-coded by engineers C) A capability that appears reliably only past a certain scale D) Always a bug **Answer: C**
13. Why did rule-based NLP systems struggle to scale?
    A. Language is finite and simple B) Language is ambiguous, productive, and drifts over time C) Rules are always faster than statistics D) They had too much training data **Answer: B**
14. Bias in a pretrained model primarily comes from:
    A. The programming language used B) Statistical patterns present in the training data C) The GPU vendor D) Model file compression **Answer: B**
15. Hallucination refers to:
    A. A model crashing B) Fluent but factually incorrect output C) A tokenizer error D) Slow inference speed **Answer: B**
16. Does scaling a model up guarantee reduced bias?
    A. Yes, always B) No — scale doesn't guarantee reduced bias C) Only for encoder-only models D) Only if trained in one language **Answer: B**
17. Self-supervised learning means:
    A. Labels come from human annotators only B) Labels are automatically derived from the data itself C) No training occurs D) Only images can be used **Answer: B**
18. "General-purpose" in the context of an LLM means:
    A. One model can be applied across many different tasks B) The model only works on one task C) The model has no parameters D) The model cannot be fine-tuned **Answer: A**
19. In "The trophy didn't fit in the suitcase because it was too big," attention should mostly connect "it" to:
    A. "suitcase" B) "trophy" C) "fit" D) "big" **Answer: B**
20. A small fine-tuned encoder model is often preferred over a large decoder-only LLM API for narrow classification tasks because of:
    A. Higher latency and cost B) Lower latency/cost and more predictable behavior at scale C) Inability to be evaluated D) Lack of any practical use **Answer: B**

---

14. Hands-On Exercises

Exercise 1 — Task Mapping List five NLP-powered features used regularly (spam filter, translate button, search, autocomplete, moderation). For each, identify the classic task and the best-fit architecture family (encoder / decoder / encoder-decoder).

Exercise 2 — Pipeline Walkthrough Without writing code, describe the exact three-stage flow a `pipeline("summarization")` call would perform, start to finish.

Exercise 3 — Attention by Hand Take the sentence "The trophy didn't fit in the suitcase because it was too small." Which word should "it" now attend to most strongly, and why does the answer flip compared to the "too big" version?

Exercise 4 — Pretrain vs. Fine-Tune Judgment Call Given 500 labeled examples for a new internal classification task, write two sentences arguing for prompting a pretrained model first, and two sentences on when fine-tuning would actually make sense instead.

Exercise 5 — Bias Spot-Check Pick any public text-generation demo. Prompt it with a role-based sentence (e.g., "Write two sentences about a nurse" and "Write two sentences about a CEO") and note any default assumptions it makes. Write down what to test before shipping something similar in a real product.

Exercise 6 — Architecture Speed Drill For each of the following, name the architecture family to start with, in one sentence: (1) real-time chat moderation, (2) a coding autocomplete tool, (3) meeting-transcript summarization, (4) internal wiki semantic search.

---

15. Revision Checklist

- ☐ Explain, in one sentence, the difference between NLP and an LLM.
- ☐ Name the three stages of the `pipeline()` abstraction.
- ☐ List at least six classic NLP task types and give an example of each.
- ☐ Explain why zero-shot classification doesn't need task-specific training data.
- ☐ Explain, at a high level, why Transformers replaced RNNs (two reasons, not one).
- ☐ Name the founding Transformer paper's title and its approximate publication year.
- ☐ Explain the difference between pretraining and fine-tuning, including why fine-tuning is cheaper.
- ☐ Define catastrophic forgetting and one way to guard against it.
- ☐ Explain, with an example sentence, what attention actually computes.
- ☐ Map a new task to encoder-only / decoder-only / encoder-decoder without hesitating.
- ☐ Explain where model bias actually comes from, in one sentence.
- ☐ Explain the difference between hallucination and bias.
- ☐ Name at least 10 terms from the glossary from memory.
- ☐ Score at least 16/20 on the mini quiz without looking back.

> Re-do the mini quiz cold after a couple of weeks. A lower score the second time is a signal to revisit this notebook — spaced repetition is supposed to feel a little harder the second time.

Reference: Hugging Face LLM Course — Chapter 1